Challenge 1 – AI Security

Analysis and Security Hardening of an AI-Assisted Access Control API

Team: Team03
Verification Date: October 25, 2026
Scope: Local Python laboratory only

1. Introduction

The lab has developed a Flask API that processes an image using a facial recognition model and an object detection model to determine whether physical access is authorized. The facial recognition model consistently returns “AUTHORIZED” with a confidence level of 0.98. The object detector, meanwhile, identifies the word “KNIFE” with a confidence level of 0.88. This allows us to test security decisions without the need for real data or systems.

The analysis follows the PROVEN → DEFEND → RETEST cycle. The results presented in this document pertain to the tests actually performed within the project folder.

2. Scenario and Security Objective

We see that the main advantage is the management of access decisions. We also notice strengths such as, the reliability of models, the structure of models, the integrity of log repository, the availability of service and the reliability of information.

A secure decision must deny access when an image is invalid, when a model fails, when a result is inconsistent or when a dangerous object is detected.

3. Architecture and Attack Surface

```
[local client / curl / test_client]
                |
                | multipart image (untrusted)
                v
[Flask POST /api/access]  -- T1 upload/HTTP boundary
                |
        decode + preprocess
                |
                v
[face model + object model]
                |
        fail-closed decision
                |
                v
[minimal JSON] + [logs/secure_uploads]
```

Untrusted inputs include the HTTP method, request size, filename, image bytes, and model outputs. The original code, models with known fingerprints, and decision policy are trusted assets.

4. Model Preparation and Compatibility

The dependencies available, in.venv were verified: Python 3.14.7, NumPy 2.5.4 Flask 3.1.3 and OpenCV 5.0.0.93.

Both pickle files are 55 bytes and reference __. _Main__.MockObjectDetectionModel.

A direct import of lab_access_api produced the error:

AttributeError: module '__main__' has no attribute 'MockFaceRecognitionModel'

Running the file as a script is the execution mode expected by the model generator. The test runner therefore loads the baseline under ____ to reproduce the laboratory environment without modifying the models.

The secured version retains the classes verifies their SHA-256 hashes and then uses a RestrictedModelUnpickler limited to the two known classes.

5. Red Team Analysis of the Original Code

The main vulnerabilities are:

1. `Preprocess_image` calls `cv2.imdecode` without validating the buffer or the result, then returns an image without resizing, normalization, or batch dimensions.

2. The route calls the `file.save()` and `file.read()` functions. Since the stream has already reached its end, OpenCV receives the bytes.

3. The policy only checks the weapon label even though the model returns "knife." The threshold of 0.15 is not appropriate, for a physical access control barrier.

4. The filename provided by the client is concatenated with./logs/.

5. Pickle.load() is not restricted.

6. The response exposes confidence scores, detected objects, the model path, the IP address, the version and the log path.

7. There is no request size limit and app.run(... Debug=) enables debugging when the application starts.

8. Exceptions are not converted to controlled errors. The decision is not explicitly terminated in the event of a failure.

The vulnerability matrix and their impacts are summarized in the table and in the JSON evidence files stored in `evidence/`.

6. Controlled Red Team Tests

The most recent data is stored in evidence/baseline_controlled_run_final.json. Previous raw HTTP data is stored in evidence/evidence/first_error.txt and evidence/second_error.txt.

Test A. POST without an image

• Reference result: HTTP 400.

• Response: {“error”:“No image provided”}.

Test B. Synthetic 300 × 300 PNG

• Expected result: HTTP 500 error when using the test client.

• Previous HTTP evidence shows that the error is caused by an empty buffer.

Test C. Text file

• Expected result: OpenCV HTTP 500 error, instead of a controlled rejection.

Preprocessing test

• A direct call using a PNG produced an image with dimensions (300,300,3) and type uint8.

• The model rejected this input format.

Isolated Policy Test

• The expected tensor was temporarily injected to observe the execution path.

• The system returned `access=true` despite detecting `knife:0.88`.

• Internal details were also revealed.

Controlled Path Test

• File name: ../logs/…..png.

• The test remained confined to the lab environment.

• The path provided by the client was included in the response. The file remained in the local directory used for testing.

The policy instrumentation has been explicitly isolated. It does not replace the results of tests B and C; rather, it provides access to the part of the code that would otherwise be hidden by the flow-handling bug.

7. Blue Team Defenses

lab_access_api_secure.py file does the following things:

• The request size is limited to 1 MiB and the image size is limited to 900 octets.

• Only a single in-memory read is allowed. The image is decoded with validation. Any OpenCV error becomes an HTTP 400 response.

• The image is resized to 224 × 224 converted to RGB turned to float32 normalized to the range [0,1] and formatted into a tuple of (1, 224 224 3).

I want to make clear that the words knife, firearm, weapon, weapon variants and explosives are considered dangerous. The confirmation threshold is 0.80. Even a tiny doubt leads to rejection.

The minimal required facial confidence level is 0.90. Unknown labels and malformed results are rejected.

Files are named with UUIDs in logs/secure_uploads. The parent directory is verified.

Responses are minimal; diagnostics are available, on the server.

SHA-256 hashes are used. A restricted deserializer is employed to ensure compatibility with simulated models. In production a non-executable format or an isolated service is preferred.

Debug mode is explicitly disabled. The default host is the local loopback and error handling is generic.

Each decision is documented in lab_access_api_secure.py. Verified by the tests, in tests.

8. Retest

The results are stored in evidence/secure_controlled_retest_final_2.json.

The regression test suite was run using unittest. Nine tests were performed, and all nine passed.

An initial assertion for a request expected the multipart parser to throw an exception. Werkzeug treated the payload as a form, and the route returned “No image provided.”

The initial failure is logged in evidence/unit_test_output_final_9.txt. The test was modified to call the HTTP 404 handler. The fixed test.

The results can be found in evidence/unit_test_output_final_9_fixed.txt.

Baseline vs. Secured Version

Test	Baseline	Secured Version
A — No image	HTTP 400	HTTP 400
B — Valid PNG	HTTP 500 before reaching the model	HTTP 200, access:false because knife:0.88 is rejected
C — Text file	OpenCV HTTP 500 / traceback in HTTP evidence	HTTP 400, generic message without a traceback
D — Request larger than 1 MiB	No configured limit	HTTP 413, request rejected
Preprocessing	(300,300,3) uint8	(1,224,224,3) float32, values in [0,1]
Response	Paths, IP address, scores, and labels exposed in the isolated execution path	Only {"access":false}
Filename	Client-supplied path used	UUID filename directly under logs/secure_uploads
Models	Unrestricted pickle.load(), incompatible direct import	Verified hashes and authorized classes loaded

An attempt to perform a direct network startup was also made in the test environment. The operation failed with the message “Unauthorized operation.” This error is logged in the file evidence/startup_probe_2026-10-25.txt.

Consequently, the validation tests reported here were performed using the Flask test client rather than through a supposedly functional network listener.

9. Residual Risks and Recommendations

•Pickle is still a serialization format. The current setup works fine for the two fixed lab models. It won't work for any random or unknown models.

•In a deployment the embedded hashes should be replaced with a signed and protected trust chain. That's how you make sure things are secure and trustworthy.

•Flask's request size limits don't cover everything. You still need rate limiting, quotas, timeouts, authentication, authorization and monitoring. Those are all important, for security.

• The simulated models don't show how well they work with biometrics or how they handle adversarial changes. For use you'd need test data, proper thresholds, human check-ins and full security tests.

•The local demo log isn't an audit trail. In production you must set access rules rotate logs regularly and follow retention rules.

10. Conclusion

We looked at the PROVE phase. We saw that it showed weaknesses in input validation, stream handling, model input contracts, dangerous-object policies, model loading, file paths, information disclosure and configuration.

We also examined the DEFEND phase. We found that it tackles these problems in a separate file while leaving lab_access_api.py unchanged.

We checked the RETEST phase. We saw that it confirms the rejection of knife inputs, invalid data and oversized inputs that the tensor contract is now correct and that response information has been shortened.

Finally we noted that sandbox limits and observed errors remain documented not converted into claims of success.

Local References

- lab_access_api.py and generate_models.py: code provided or adapted from the laboratory.
- lab_access_api_secure.py: Blue Team implementation.
- evidence/baseline_controlled_run_final.json and evidence/secure_controlled_retest_final_2.json.
- evidence/before_after_comparison_final.json: machine-generated comparison of statuses.
- Materials provided under files/.